



RV College of
Engineering®

Go, change the world

MedInsure Health Claim Approval Model – Pre-Production Failure Analysis & Solutions

Machine Learning
operations



Case Study – MedInsure Health Claim Approval Model

MedInsure, a health-insurance company, has developed a machine-learning model to automatically score and approve low-risk claims under ₹25,000. The model was trained using five years of historical claim records, hospital types, diagnosis codes, treatment cost ranges, customer risk profiles, and fraud-flag indicators. During development, the model achieved strong accuracy and passed internal validation. Now, the company plans to move the model to the production environment.

Issues Encountered During Pre-Production Testing:

1. Runtime Environment Mismatch:

Predictions from the staging server differ significantly from the development notebook. The staging environment uses Python 3.10, while the model was trained using Python 3.8, and several ML library versions differ.

2. *Data Access & Schema Changes:*

In staging, the “hospital_risk_score” feature is missing because the ETL pipeline was updated last month, but the model still expects this feature. As a result, the model rejects 22% of claim requests.

3. *Model Risk Evaluation Concerns:*

The auditor notices the model performs poorly for claims involving “rare surgical procedures” due to insufficient representation in the training data.

4. *Quality Assurance & Stress Testing:*

Under high load (5,000 claims/min), the API slows down, causing timeouts for the customer-facing mobile app.

5. *Reproducibility Issues:*

The auditor requests proof of training data versions and hyperparameters. The data science team cannot fully reproduce the exact training run because environment logs were not captured.

6. *Security & Adversarial Inputs:*

Fraud analysts discovered that attackers manipulate diagnosis codes inside claims to trick the model into classifying high-risk claims as low-risk.



Identify the sources of model risk in this scenario. How does poor performance on rare surgical claims illustrate the importance of evaluating model robustness across

1. Data Risk:

Missing or incomplete representation of “rare surgical procedures”.
Missing feature (“hospital_risk_score”) in staging due to schema change.
Model heavily dependent on specific features → brittle.

2. Model Bias & Generalization Risk

The model learned patterns mostly from common procedures.
Rare cases are underrepresented in training → poor prediction accuracy.
This creates hidden bias against smaller subgroups.

3. Operational Risk

Environment mismatch (Python 3.8 vs 3.10) causes different outputs.
API failures at high load (5,000 claims/min) → unstable model behavior.

4. Security / Adversarial Risk

Attackers manipulate diagnosis codes to get false “low-risk” scores.
Input vulnerabilities expose financial risk.

5. Reproducibility & Governance Risk

No logs of hyperparameters or training data version.
Cannot reproduce original model → audit compliance failure.



7. Based on all the issues identified, assess whether the model is ready for production. List the mandatory fixes required before deployment.

1. Fix Environment & Version Mismatch

Problem: Dev used Python 3.8; staging uses 3.10 → inconsistent predictions.

Mandatory Fixes:

Standardize a single environment across dev–staging–prod
Freeze versions using requirements.txt / conda.yml
Package model with Docker to avoid any runtime variation
Enable CI/CD checks for dependency drift

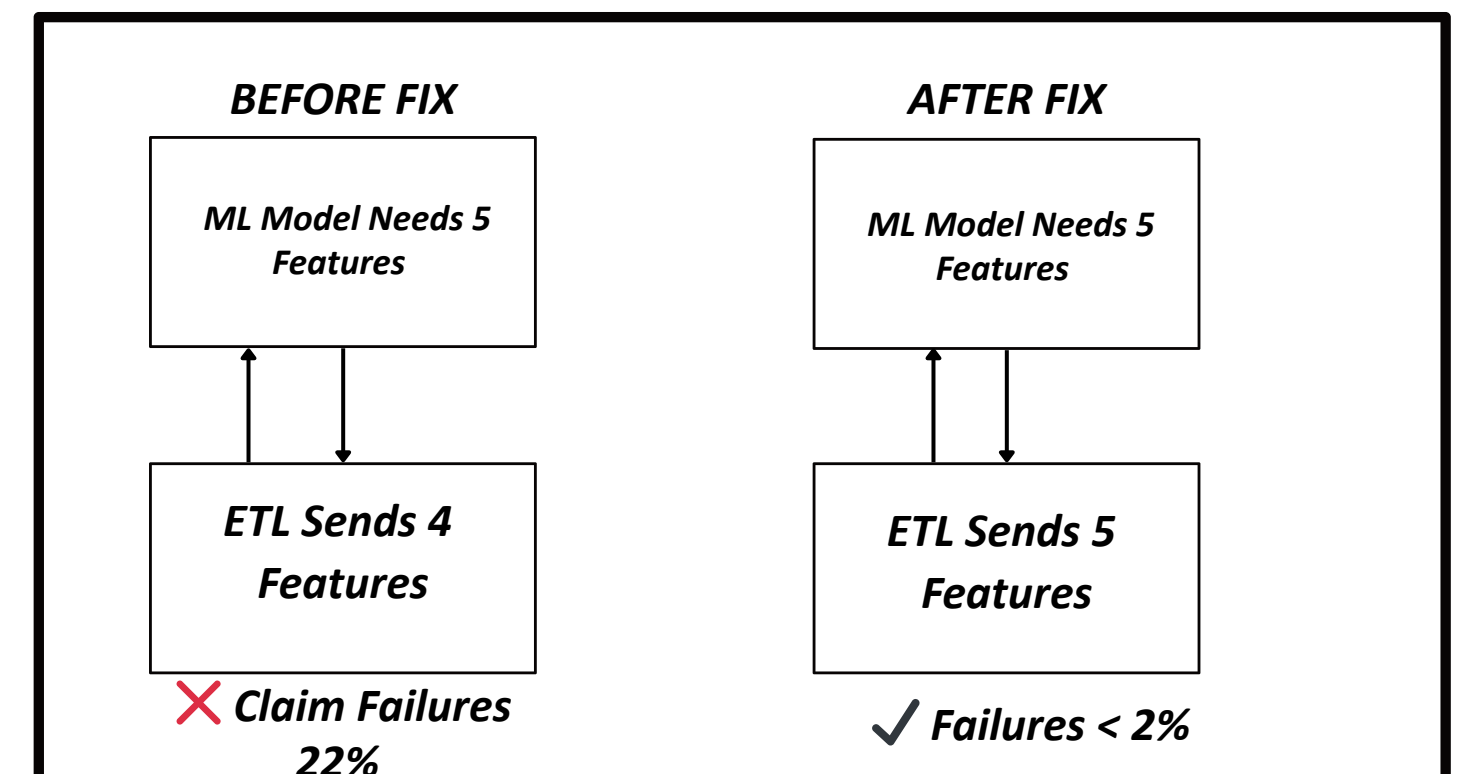
DEV ENV (Py 3.8)	STAGING ENV (Py 3.10)
<i>pandas 1.4.0</i> <i>sklearn 1.0</i> <i>numpy 1.23</i>	<i>pandas 2.0.1</i> <i>sklearn 1.4</i> <i>numpy 1.23</i>
<i>Different library behavior</i> <i>Different model output</i>	

2. Restore or Redesign the Data Schema

Problem: Missing feature (hospital_risk_score) causing 22% request failures.

Mandatory Fixes:

Implement data schema validation during inference
Update ETL pipeline OR retrain model without this feature
Create strict data contracts between ETL and ML teams
Add imputation/default value logic if the feature must be optional





2. Based on all the issues identified, assess whether the model is ready for production. List the mandatory fixes required before deployment.

3. Address Bias & Poor Performance on Rare Surgical Procedures

Problem: Very low accuracy for rare procedures due to lack of training samples.

Mandatory Fixes:

Add more historical samples / domain-augmented data

Apply class-weight adjustments or data balancing techniques

Add rule-based or human-review override for rare procedures

Re-evaluate model risk and fairness before approval

COMMON CASES (High accuracy)



RARE CASES (Low accuracy)



4. Improve Performance Under High Load

Problem: API slows down under 5,000 claims/min → app timeouts.

Mandatory Fixes:

Optimize model inference (ONNX Runtime / TF Serving / TorchServe)

Add horizontal autoscaling (Kubernetes / load balancers)

Precompute heavy features in ETL to reduce inference cost

Stress test API until it meets latency SLAs



2. Based on all the issues identified, assess whether the model is ready for production. List the mandatory fixes required before deployment.

5. Ensure Full Reproducibility

Problem: Training cannot be reconstructed;
no logs of data or hyperparameters.

Mandatory Fixes:

Use MLflow / DVC / Weights & Biases to log:
dataset versions

hyperparameters

environment & code versions

model artifacts

Establish an end-to-end reproducible training pipeline

Store environment snapshots (Dockerfile / Conda lock)

6. Build Security Protection Against Adversarial Inputs

Problem: Attackers manipulate diagnosis
codes to bypass risk scoring.

Mandatory Fixes:

Add strong input validation

(e.g., allowed medical code ranges)

Use anomaly detection or rule-based
fraud checks

Monitor model for adversarial patterns

Introduce human review for suspicious claims

Conduct adversarial robustness testing before launch

Thank you